# Words and Tokens

## Tópicos em Fundamentos e Métodos da Computação

Prof. Dr. Luís Paulo Faina Garcia

Universidade de Brasília
Departamento de Ciência da Computação

# How many words in a sentence?

They picnicked by the pool, then lay back on the grass and looked at the stars.

- 16 words if we don’t count punctuation marks as words
- 18 if we count punctuation

# How many words in a sentence?

I do uh main- mainly business data processing

- The broken-off word *main-* is called a **fragment**
- Words like *uh* and *um* are called **fillers** or **filled pauses**
- Should we consider these to be words?

# How many words in a sentence?

- **Types** are the number of distinct words in a corpus; if the set of words in the vocabulary is V, the number of types is the vocabulary size |V|
- **Word instances** are the total number N of running words.

# How many words in a sentence?

They picnicked by the pool, then lay back on the grass and looked at the stars.

- Ignore punctuation: 14 types and 16 instances
- More questions: Are *They* and *they* the same word?

# How many words in a sentence?

I'm: Orthographically one word (in the English writing system)

But grammatically two words:

- the subject pronoun I
- the verb ’m, short for am

# How many words in a sentence?

- Not every written language uses spaces!!
- Chinese, Japanese and Thai don't!

# How to choose tokens in Chinese

- Chinese words are composed of characters called "hanzi" (or sometimes just "zi")
- Each one represents a meaning unit called a **morpheme**
- Each word has on average 2.4 of them. But deciding what counts as a word is complex and not agreed upon

# How to choose tokens in Chinese

姚明进入总决赛
Yao Ming reaches the finals

| | | | |
|---|---|---|---|
| 姚明 | 进入 | 总决赛 | |
| YaoMing | reaches | finals | → Chinese Treebank |
| 姚 | 明 | 进入 | 总 | 决赛 | |
| Yao | Ming | reaches | overall | finals | → Peking University |
| 姚 | 明 | 进 | 入 | 总 | 决 | 赛 | |
| Yao | Ming | enter | enter | overall | decision | game | → Just use characters |

# Tokenization across languages

- In Chinese we use characters (zi) as tokens
- But that doesn't work for, e.g., Thai and Japanese
- These differences make it hard to use words as tokens

# Too many words!

| | Types = \|V\| | Instances = N |
|---|---|---|
| Shakespeare | 31 thousand | 884,000 |
| Brown Corpus | 38 thousand | 1 million |
| Switchboard conversations | 20 thousand | 2.4 million |
| COCA | 2 million | 440 million |
| Google N-grams | 13+ million | 1 trillion |

The bigger the corpus, the more word types!

# Too many words!

Heaps Law: Vocabulary size for a text goes up with the square root of its length in words

$$|V| = kN^{\beta} \longrightarrow \sim 0.5$$

N = number of instances

|V| = number of types in vocabulary V

k and β are positive constants, and 0 < β < 1.

# Two kinds of words

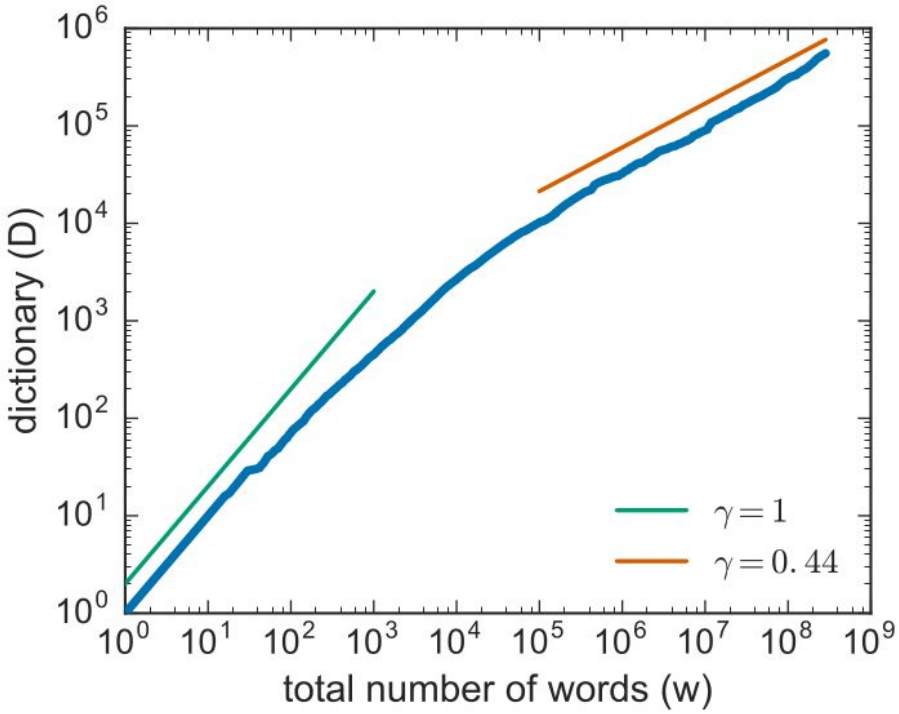
**Function words:** connectors

- of, the, is, and, una, e, 是

**Content words:** nouns, adjectives and verbs

- mango, braise, snowy, feliz, 北京

# Too many words!



# Too many words!

- No matter how big our vocabulary, there will always be words we missed! We will always have unknown words!
- **Unknown words:** words that it has never seen before.

# Words and Subwords

Because of these two problems:

1. Many languages don't have **orthographic words**
2. The number of words grows without bound

NLP systems don't use words, but smaller units called **subwords** that can be recombined to model new words

To think about defining **subwords**, we first need to talk about **units** that are smaller than words; **morphemes** and **characters**.

# Morphemes: Parts of Words

**Morpheme:** a minimal meaning-bearing unit in a language

- fox: one morpheme
- cats: two morphemes *cat* and *-s*

# Morphemes: Parts of Words

Here's a sentence in English segmented into morphemes with hyphens:

Doc work-ed care-ful-ly wash-ing the glass-es.

# Types of morphemes

Morphemes can be of 2 types:

- **Roots:** the central morpheme supplying the main meaning
- **Affixes:** adding additional meanings of various kinds.

Doc work-ed care-ful-ly wash-ing the glass-es.

| | | |
|---|---|---|
| worked | root:work | affix:-ed |
| glasses | root:glass | affix:-es |

# Types of affixes

Affixes also have 2 types:

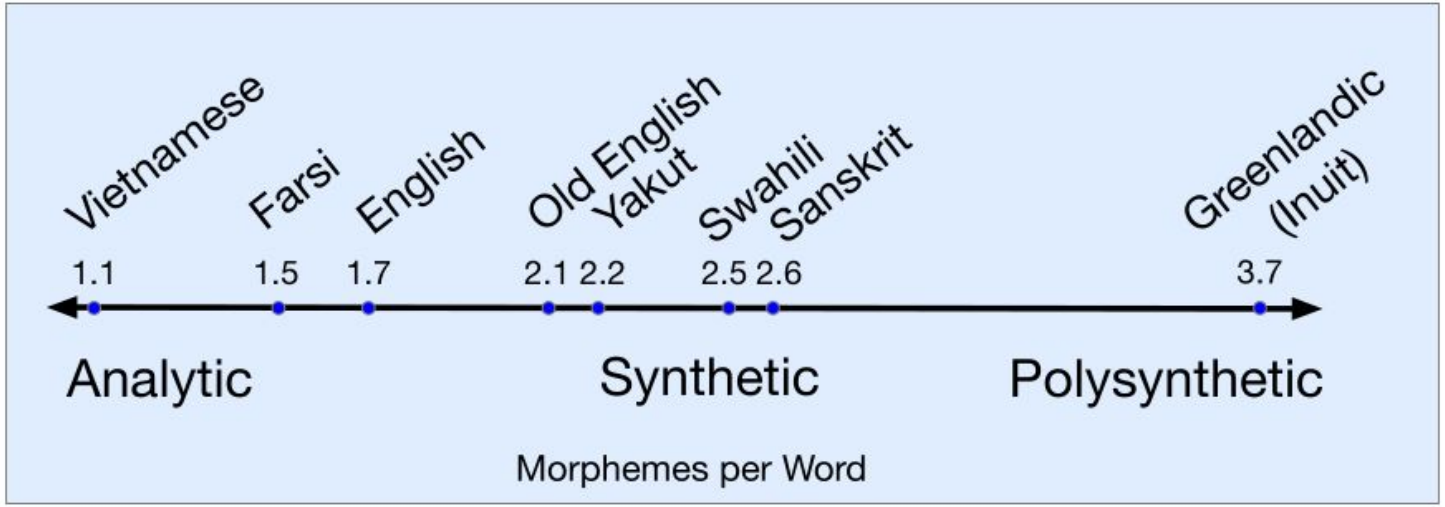
- **Inflectional morphemes:** are grammatical morphemes that tend to play a syntactic role, such as marking agreement.
  - -ed past tense on verbs
  - -s/-es plural on nouns
- **Derivational morphemes:** apply only to a specific subclass of words and result in a word of a different grammatical class
  - care (noun) careful (adjective) carefully (adverb)

# Morphological Typology

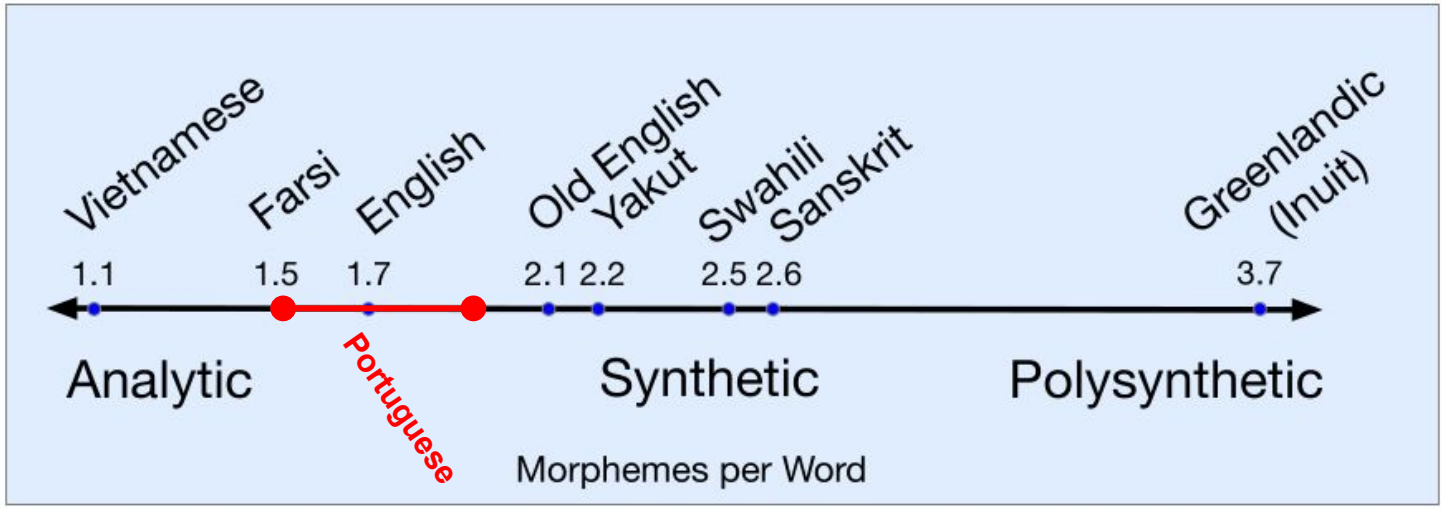
The study of how languages vary in their morphology, i.e., how words break up into their parts, is called **morphological typology**.

- Two dimensions are salient for tokenization:
  - Number of morphemes per word
  - How easy it is to segment the morphemes

# Morphemes per word



# Morphemes per word




**Depende do tipo de documento em Português**

# How easily segmentable

**Agglutinative languages:** Very clean boundaries between morphemes (Turkish)

**Fusion languages:** A single affix may conflate multiple morphemes

She reads the article.

Means both "third person" and "present tense"

# Unicode

The Unicode standard is a method for representing text written using any character  (more than 150.000) in any script of the languages of the world (168 to date).

- Chinese, Arabic, Hindi, Cherokee, Ethiopic, Khmer, N’Ko,…
- Dead ones like Sumerian cuneiform
- Invented ones like Klingon
- Plus emojis, currency symbols, etc.

# ASCII

Starting in the 1960s, the Latin characters used to write English, were represented with a code called ASCII (American Standard Code for Information Interchange)

- ASCII represented each character with a single byte
- A byte (8 bits) can represent 256 different characters
- ASCII only used 127 of them (7 bits is enough)
- However only 95 used
- Teletypes for the rest

# ASCII

This sentence has non-ASCII ñ and ó

```
Señor- respondió Sancho
```

And lots of languages aren’t based on Latin characters at all!

अनुच्छेद १(एक): सभी मनुष्य जन्म से स्वतन्त्र तथा मर्यादा और अधिकारों में समान होते हैं। वे तर्क और विवेक से सम्पन्न हैं तथा उन्हें भ्रातृत्व की भावना से परस्पर के प्रति कार्य करना चाहिए।

Hindi text of the Universal Declaration of Human Rights

# Code Points

Unicode assigns a unique ID (code point) to each of its 150.000 characters:

- Written in hex, with prefix U+
  - a is `U+0061` which is `0x0061`
- First 127 code points is ASCII
  - For backwards compatibility

# Code Points

- A code point has no visuals.
- It is not a glyph (the visual representation of a character).
- The code point is a abstract representation
- Glyphs are stored in fonts.

| | | |
|---|---|---|
| 0061 | a | LATIN SMALL LETTER A |
| 0062 | b | LATIN SMALL LETTER B |
| 0063 | c | LATIN SMALL LETTER C |
| 00F9 | ù | LATIN SMALL LETTER U WITH GRAVE |
| 00FA | ú | LATIN SMALL LETTER U WITH ACUTE |
| 00FB | û | LATIN SMALL LETTER U WITH CIRCUMFLEX |
| 00FC | ü | LATIN SMALL LETTER U WITH DIAERESIS |
| 8FDB | 进 | |
| 8FDC | 远 | |
| 8FDD | 违 | |
| 8FDE | 连 | |
| 1F600 | 😀 | GRINNING FACE |
| 1F00E | 🀎 | MAHJONG TILE EIGHT OF CHARACTERS |

# Encoding and UTF-8

- We need to represent a character in a text string so we write an encoding of the character.
- There are many different possible encoding methods, but the encoding method called UTF-8 is by far the most frequent

# Encodings

- Hello has 5 code points
- How to write in a file?
- There are more than 1 million code points
- So would need 4 bytes
- But that would make files very long!

| h | e | l | l | o |
|---|---|---|---|---|
| U+0068 | U+0065 | U+006C | U+006C | U+006F |

# Variable Length Encoding

- Unicode Transformation Format 8 (UTF-8)
- For the first 127 code points, same as ASCII
  - UTF-8 encoding of hello is : 68 65 6C 6C 6F
- Code points ≥128 are encoded as a sequence of 2, 3, or 4 bytes
  - In range 128 - 255, so won't be confused with ASCII
  - First few bits say if its 2-byte, 3-byte, or 4-byte

# Variable Length Encoding

| Code Points | | UTF-8 Encoding | | | |
|---|---|---|---|---|---|
| **From - To** | **Bit Value** | **Byte 1** | **Byte 2** | **Byte 3** | **Byte 4** |
| U+0000-U+007F | 0xxxxxxx | xxxxxxxx | | | |
| U+0080-U+07FF | 00000yyy yyxxxxxx | 110yyyyy | 10xxxxxx | | |
| U+0800-U+FFFF | zzzzyyyy yyxxxxxx | 1110zzzz | 10yyyyyy | 10xxxxxx | |
| U+010000-U+10FFFF | 000uuuuu zzzzyyyy yyxxxxxx | 11110uuu | 10uuzzzz | 10yyyyyy | 10xxxxxx |

ñ code point `U+00F1` or bit sequence `00000000 11110001`

Gets encoded with pattern `110yyyyy 10xxxxxx`

So is mapped to a two-byte bit sequence

`11000011 10110001 = 0xC3B1`

# Variable Length Encoding

- The first 127 characters (ASCII) map to 1 byte
- Most remaining characters in European, Middle Eastern, and African scripts map to 2 bytes
- Most Chinese, Japanese, and Korean characters map to 3 bytes
- Rarer characters, emojis/symbols map to 4 bytes.

# UTF-8 encoding

- Backwards compatible with ASCII
- Efficient: fewer bytes for common characters
- Doesn't use zero bytes (except for NULL character U+0000)
- If a file is corrupted, the nearest character boundary is always findable by moving only up to 3 bytes

# UTF-8 and Python

- Python strings stored internally as Unicode
  - Each string a sequence of Unicode code points
  - String functions, regex apply natively to code points.
- Files need to be encoded/decoded when written or read
  - Every file is stored in some encoding
  - If it's not UTF-8 it's something older like ASCII or ISO8859.1

# Standard for tokenization

Tokenization is the first stage of natural language processing. It is the process of segmenting the running input text into tokens.

Instead of

- White-space or orthographic words
- Unicode characters
- Morphemes

We use the data to tell us how to tokenize!

# Why tokenize?

- Using a deterministic series of tokens means systems can be compared equally
- Algorithms like Language Models assume all texts have a fixed tokenization
- Tokenization algorithms that include smaller tokens for morphemes eliminate the problem of unknown words.
- If some word occurs in test set but not training set, we still know how to segment it into known tokens.

# Subword tokenization

- Two most common algorithms:
  - Byte-Pair Encoding (BPE)
  - Unigram or Sentence Piece tokenizer
- All have 2 parts:
  - **Token learner:** takes a raw training corpus and induces a vocabulary
  - **Token encoder:** takes a raw test sentence and tokenizes it according to that vocabulary

# Byte Pair Encoding (learner)

- Iteratively merge frequent neighboring tokens to create longer tokens.

```
function BYTE-PAIR ENCODING(strings C, number of merges k) returns vocab V

  V ← all unique characters in C                    # initial set of tokens is characters
  for i = 1 to k do                                 # merge tokens k times
    t_L, t_R ← Most frequent pair of adjacent tokens in C
    t_NEW ← t_L + t_R                               # make new token by concatenating
    V ← V + t_NEW                                   # update the vocabulary
    Replace each occurrence of t_L, t_R in C with t_NEW      # and update the corpus
  return V
```

# Byte Pair Encoding (learner)

Example with synthetic corpus:

```
set␣new␣new␣renew␣reset␣re
new
```

| corpus | | vocabulary |
|---|---|---|
| 2 | ␣ n e w | ␣, e, n, r, s, t, w |
| 2 | ␣ r e n e w | |
| 1 | s e t | |
| 1 | ␣ r e s e t | |

The most frequent is the pair `ne`. It occurs in `new` (2) and `renew` (2) for a total of 4 occurrences.

# Byte Pair Encoding (learner)

Example with synthetic corpus:

```
set␣new␣new␣renew␣reset␣re
new
```

| corpus | | vocabulary |
|---|---|---|
| 2 | ␣ ne w | ␣, e, n, r, s, t, w, ne |
| 2 | ␣ r e ne w | |
| 1 | s e t | |
| 1 | ␣ r e s e t | |

Now the most frequent pair is `new` (4).

# Byte Pair Encoding (learner)

Example with synthetic corpus:

```
set␣new␣new␣renew␣reset␣re
new
```

**corpus**

| | |
|---|---|
| 2 | ␣ new |
| 2 | ␣ r e new |
| 1 | s e t |
| 1 | ␣ r e s e t |

**vocabulary**

␣, e, n, r, s, t, w, ne, new

Next `r` (3) get merged to `␣r`. Then `re` (3) gets merged to `re`.

# Byte Pair Encoding (learner)

Example with synthetic corpus:

```
set␣new␣new␣renew␣reset␣re
new
```

| corpus | |
|---|---|
| 2 | ␣ new |
| 2 | ␣re new |
| 1 | s e t |
| 1 | ␣re s e t |

**vocabulary**

␣, e, n, r, s, t, w, ne, new, ␣r, ␣re

# Byte Pair Encoding (learner)

| merge | current vocabulary |
|---|---|
| (␣, new) | ␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new |
| (␣re, new) | ␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew |
| (s, e) | ␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se |
| (se, t) | ␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se, set |

# Byte Pair Encoding (encoder)

Once we've learned our vocabulary, the BPE encoder is used to tokenize a test sentence. It runs them greedily, in the order we learned them.

- First: segment each test word into characters
- Then run rules: (1) merge `n e` to `ne`, (2) merge `ne w` to `new`, etc.
- Result:
  - Recreates training set words
  - But also learns subwords like `re` and `new` that might appear in new words like `rearrange` and `anew`

# Visualizing GPT-4o tokenizer

Anyhow, ·she's·seen·Jane's·224123·flowers·anyhow!

- Clitics like ’s
  - Are segmented of Jane
  - But part of frequent words like she’s
- Numbers segmented into chunks of 3 digits
- Anyhow and ·anyhow are segmented differently

# Tokenizing across languages

- Even though BPE tokenizer are multilingual
- LLM training data is still vastly dominated by English
- Most BPE tokens used for English, leaving less for other languages
- Words in other languages are often split up

# Tokenization is better in English

English: 18 tokens; no words are split into multiple tokens:

```
In·a·deep·bowl,·mix·the·orange·juice·with·the·sugar,·g
inger,·and·nutmeg.
```

Spanish: 33 tokens; 6/16 words are split

```
En·un·recipiente·hondo,·mezclar·el·jugo·de·naranja·con
·el·azúcar,·jengibre,·y·nuez·moscada.
```

# Corpora

Words don't appear out of nowhere!

A text is produced by

- a specific writer,
- at a specific time,
- in a specific variety,
- of a specific language,
- for a specific function.

# Corpora vary along dimensions like

- Language: 7097 languages in the world
- It's important to test algorithms on multiple languages
- What may work for one may not work for another

# Corpora vary along dimensions like

- **Variety:** African American English (AAE) and Mainstream American English (MAE):
  - iont: I don't
  - talmbout: talking about
- **Genre:** newswire, fiction, scientific articles, Wikipedia
- **Demographics:** writer's age, gender, ethnicity, socio-economic status

# Corpora vary along dimensions like

**Code Switching:** Speakers use multiple languages in the same sentence. This is very common around the world especially in spoken language and social media.

# Corpus datasheets

Motivation: Why was the corpus collected?

By whom? Who funded it?

Situation: In what situation was the text written?

Collection process: How was it sampled? Was there consent? Pre-processing?

# Regular Expressions

Regular Expressions (Regex) are patterns used to search, match, and manipulate text. It is a language for specifying text strings based on symbolic syntax to describe sets of strings.

Widely used in NLP:

- Data preprocessing
- Data validation (emails, IDs, etc.)
- Information extraction

# The Square Bracket (Regex)

Square brackets [ ] define a set of possible characters

Examples:

- [abc] → matches a, b, or c
- [0-9] → any digit
- [a-zA-Z] → any letter

Negation:

- [^abc] → any character except a, b, or c

# Counting, Optionality, and Wildcards (Regex)

Quantifiers:

- * → 0 or more times
- + → 1 or more times
- ? → 0 or 1 time

Specific counts:

- {n} → exactly n times
- {n,m} → between n and m times

Wildcard:

- . → any character (except newline)

# Examples of Quantifiers (Regex)

- a+ → "a", "aa", "aaa"
- ba* → "b", "ba", "baa"
- colou?r → "color" or "colour"
- \d{3} → exactly 3 digits

# Common Shorthands (Regex)

- \d → digit → [0-9]
- \D → non-digit → [^0-9]
- \w → word character → [a-zA-Z0-9_]
- \W → non-word character → [^a-zA-Z0-9_]
- \s → whitespace (space, tab, newline)
- \S → non-whitespace

# Anchors and Boundaries (Regex)

Anchors define positions in text:

- ^ → start of line
- $ → end of line

Boundaries:

- \b → word boundary

Examples:

- ^Hello → starts with "Hello"
- world$ → ends with "world"
- \bcat\b → the word "cat" alone

# Disjunction, Grouping, and Precedence (Regex)

Disjunction (OR):

- a|b → matches a or b

Grouping:

- ( ) → groups patterns

Precedence

- Parentheses control evaluation order

Examples:

- (cat|dog) → "cat" or "dog"
- gr(a|e)y → "gray" or "grey"

# Examples (Regex)

Simple email regex:

- [a-zA-Z0-9._]+@[a-z]+\.[a-z]+

Explanation:

- Username: letters/numbers
- @ separator
- Domain + extension

# Examples (Regex)

Phone number regex:

- \(\d{2}\)\s?\d{4,5}-\d{4}

Captures:

- Area code in parentheses
- Optional space
- Number with 4 or 5 digits + hyphen

# Regex for BPE Pre-tokenization

BPE uses regex for text splitting. The goal is separate words, punctuation, and symbols to improve tokenization for language models.

```
>>> import regex as re
>>> pat = re.compile(
... # Contractions: 't and 'm are tokens
...     r"'s|'t|'re|'ve|'m|'ll|'d|"
... # Words:  sequence of Unicode letters (after optional space)
...     r" ?\p{L}+|"
... #Number: sequence of digits (after optional space)
...     r" ?\p{N}+|"
... # Punctuation: sequence of non-alphanumeric/non-space
...                           #(after optional space)
...     r" ?[^\s\p{L}\p{N}]+|"
... # whitespace
...     r"\s+(?!\S)|\s+"
... )
>>> text = "We're 350 dogs! Um, lunch?"
>>> print(pat.findall(text))
['We', "'re", ' 350', ' dogs', '!', ' Um', ',', ' lunch', '?']
>>>
```

# Conclusion

- Tokenization is essential for NLP and defines how text is processed.
- Word-based approaches face challenges like vocabulary growth and unknown words.
- Subword methods (e.g., BPE) improve generalization across languages.
- Unicode ensures consistent text representation.

# Conclusion

Key Takeaway:

- A well-designed tokenization strategy is crucial for building robust, scalable, and language-aware NLP models.

# References


- JURAFSKY, Daniel; MARTIN, James H. Speech and language processing. 3. ed. Draft. 2026. Disponível em: https://web.stanford.edu/~jurafsky/slp3/